UITS

Future will be better than the past An institution of **PHP** Family

University of Information Technology & Sciences

Assignment

Course Code : CSE0613212

Course Title : Data Structure and Algorithms 1 Lab

Course Teachers:

- Md. Azharul Karim Chowdhury Anik
- Md. Yousuf Ali

Submitted By:

Name : Sadia Sultana Prapty

ID : 04325205101010

Batch : 58

Section : 3A1

Semester : Autumn, 2026

Q1:

739. Daily Temperatures

Medium | Topics | Companies | Hint

Given an array of integers `temperatures` represents the daily temperatures, return *an array* `answer` *such that* `answer[i]` *is the number of days you have to wait after the* `i^th` *day to get a warmer temperature.* If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Example 1:

> **Input:** temperatures = [73,74,75,71,69,72,76,73]
> **Output:** [1,1,4,2,1,1,0,0]

Example 2:

> **Input:** temperatures = [30,40,50,60]
> **Output:** [1,1,1,0]

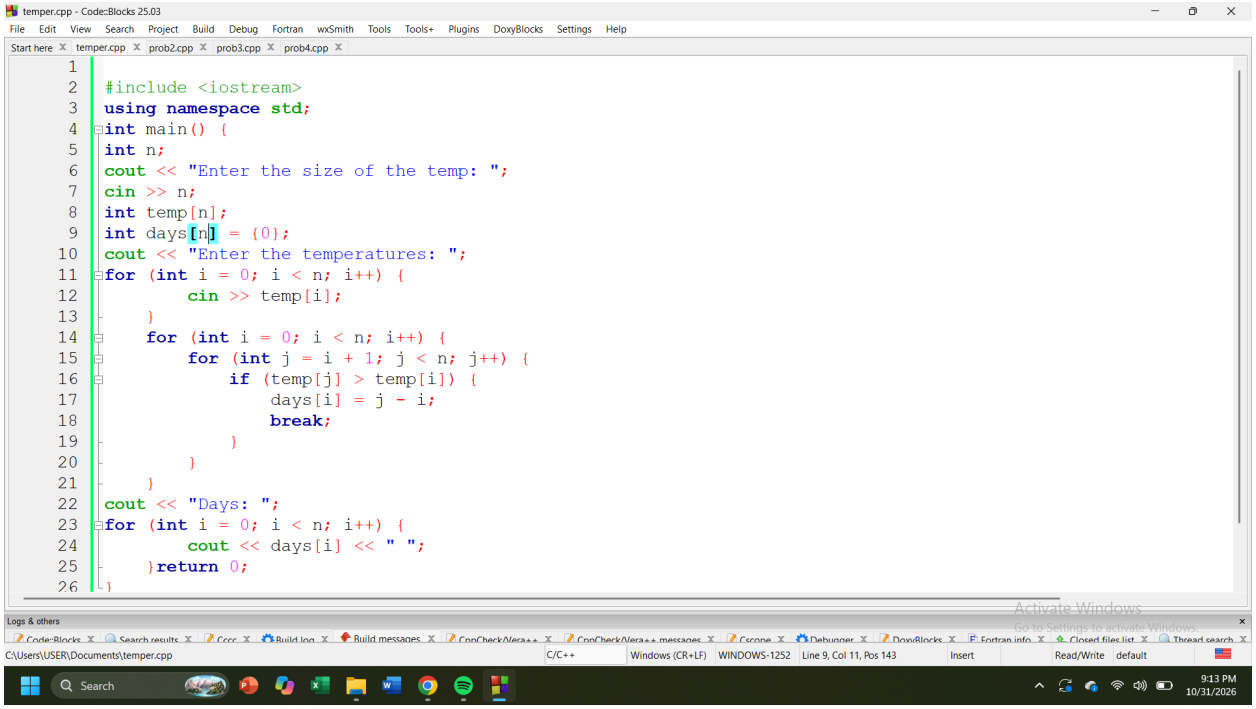
Example 3:

> **Input:** temperatures = [30,60,90]
> **Output:** [1,1,0]

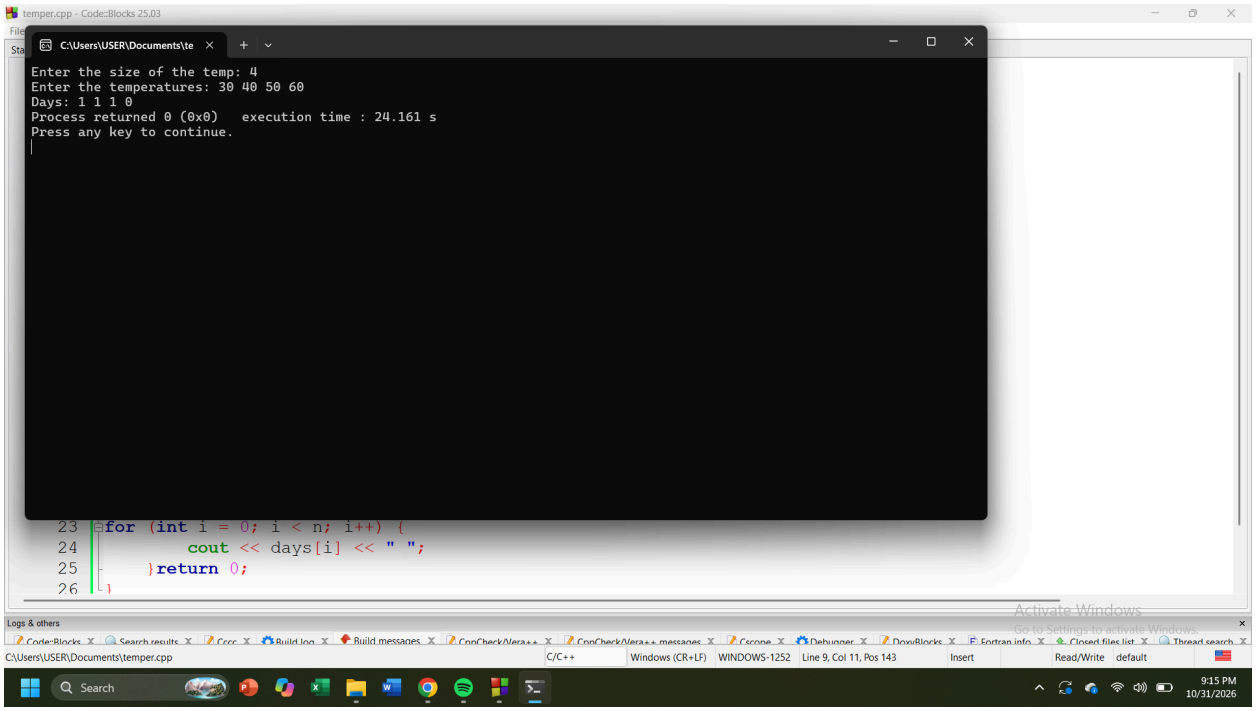
Constraints:

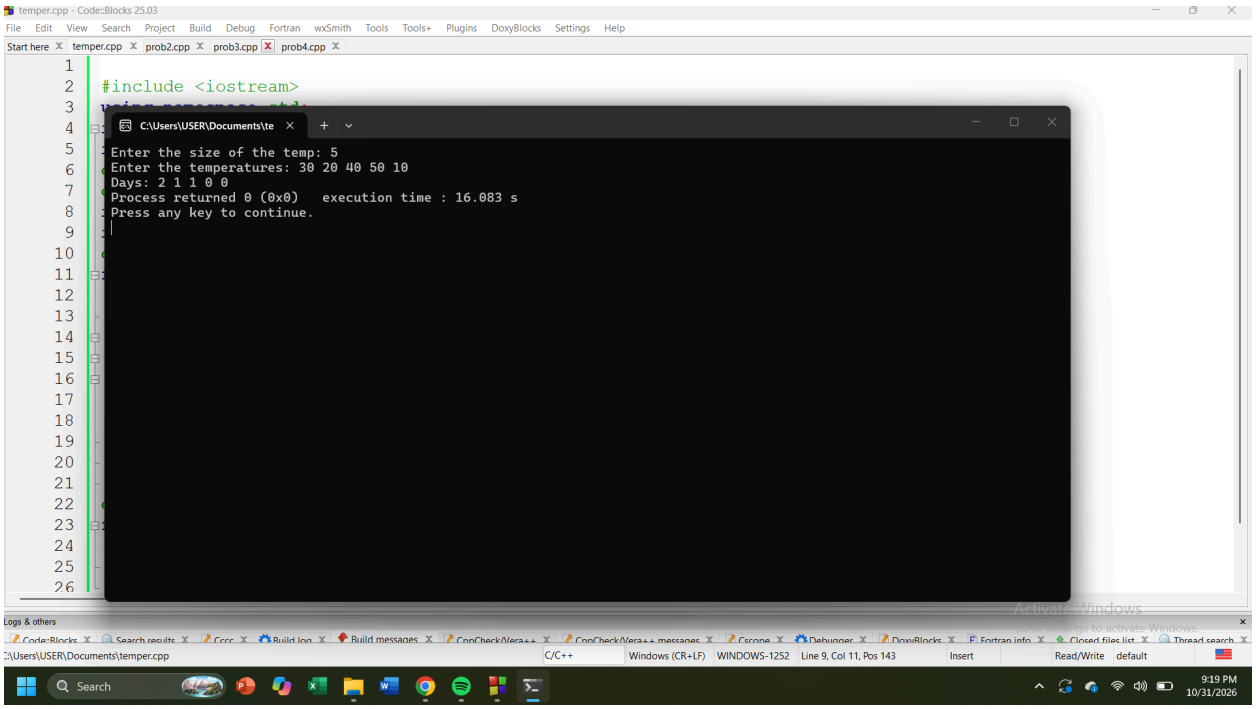
- `1 <= temperatures.length <= 10^5`
- `30 <= temperatures[i] <= 100`

Codes:



Output:


temper.cpp - Code::Blocks 25.03
C:\Users\USER\Documents\te
Enter the size of the temp: 4
Enter the temperatures: 30 40 50 60
Days: 1 1 1 0
Process returned 0 (0x0)   execution time : 24.161 s
Press any key to continue.
for (int i = 0; i < n; i++) {
cout << days[i] << " ";
}return 0;
}
Logs & others
C:\Users\USER\Documents\temper.cpp
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 9, Col 11, Pos 143
Insert
Read/Write
default
Search
9:15 PM
10/31/2026


temper.cpp - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
Start here
temper.cpp
prob2.cpp
prob3.cpp
prob4.cpp
#include <iostream>
C:\Users\USER\Documents\te
Enter the size of the temp: 5
Enter the temperatures: 30 20 40 50 10
Days: 2 1 1 0 0
Process returned 0 (0x0)   execution time : 16.083 s
Press any key to continue.
Logs & others
C:\Users\USER\Documents\temper.cpp
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 9, Col 11, Pos 143
Insert
Read/Write
default
Search
9:19 PM
10/31/2026

Q2:

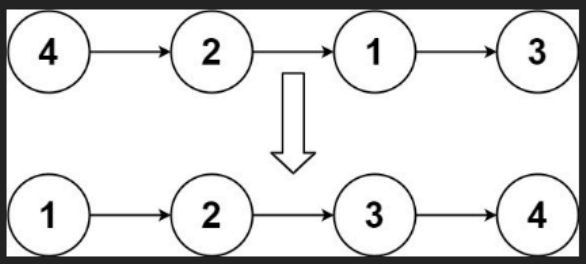
Description | Editorial | Solutions | Submissions

148. Sort List

Medium | Topics | Companies

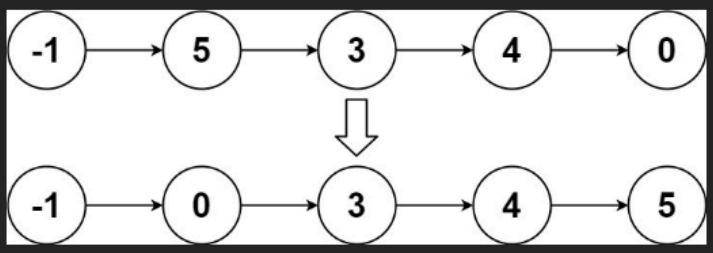
Given the `head` of a linked list, return *the list after sorting it in* ***ascending order***.

Example 1:




```
Input: head = [4,2,1,3]
Output: [1,2,3,4]
```

Example 2:



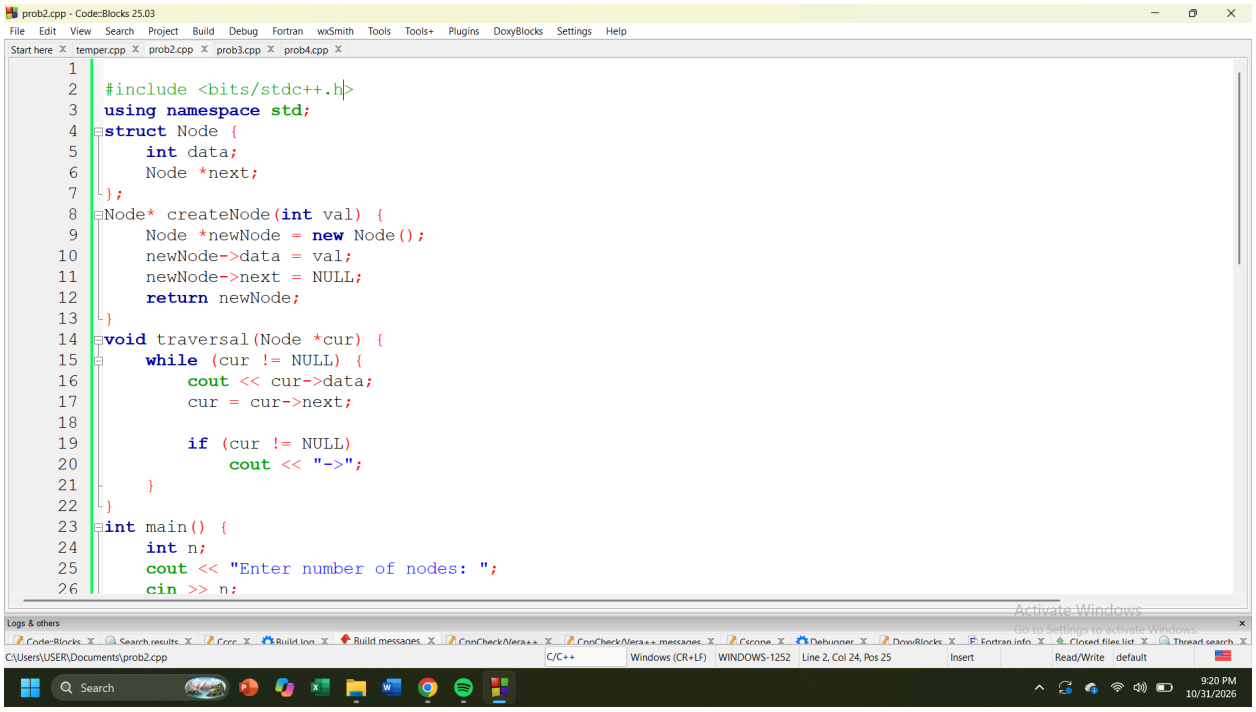

```
Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]
```

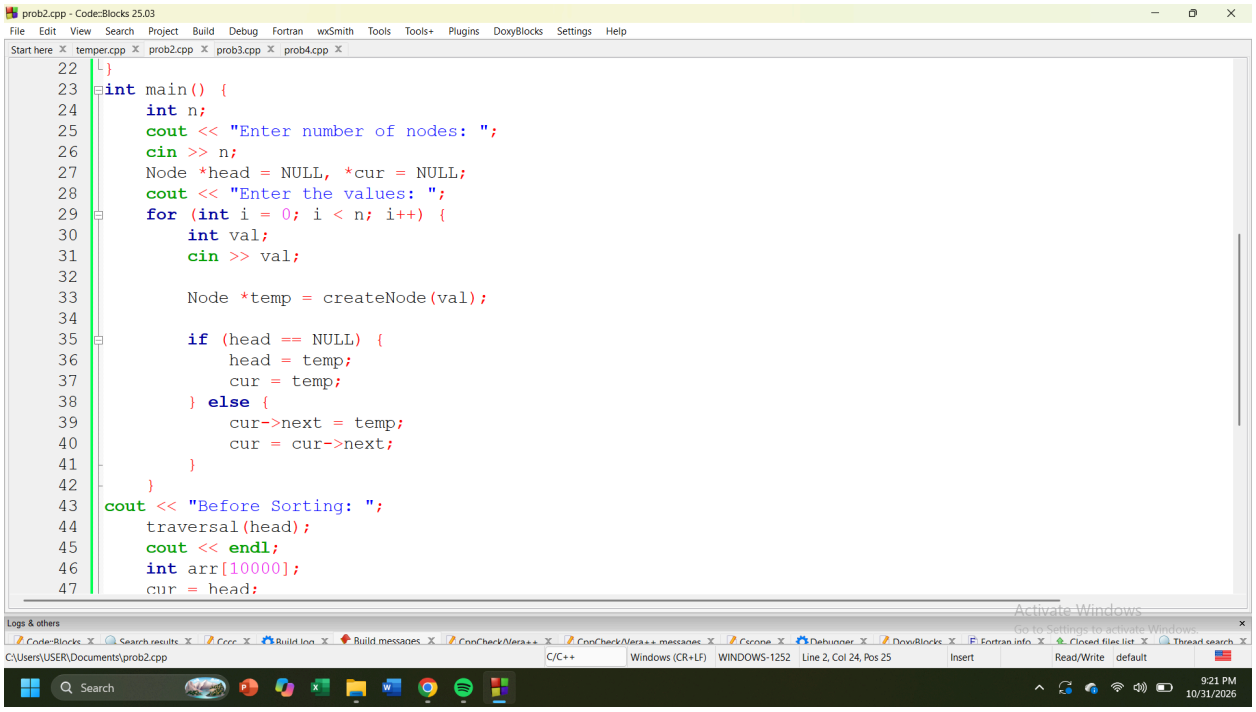
Example 3:

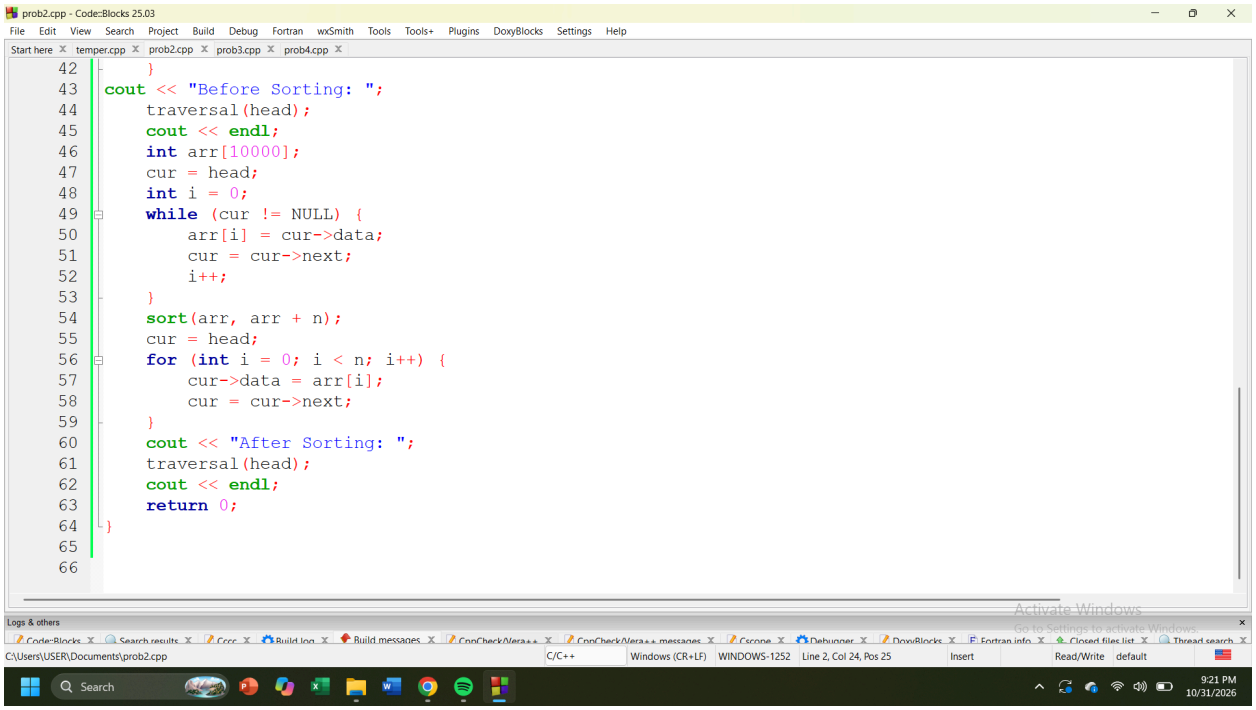
```
Input: head = []
```

13.3K | 213

Codes:

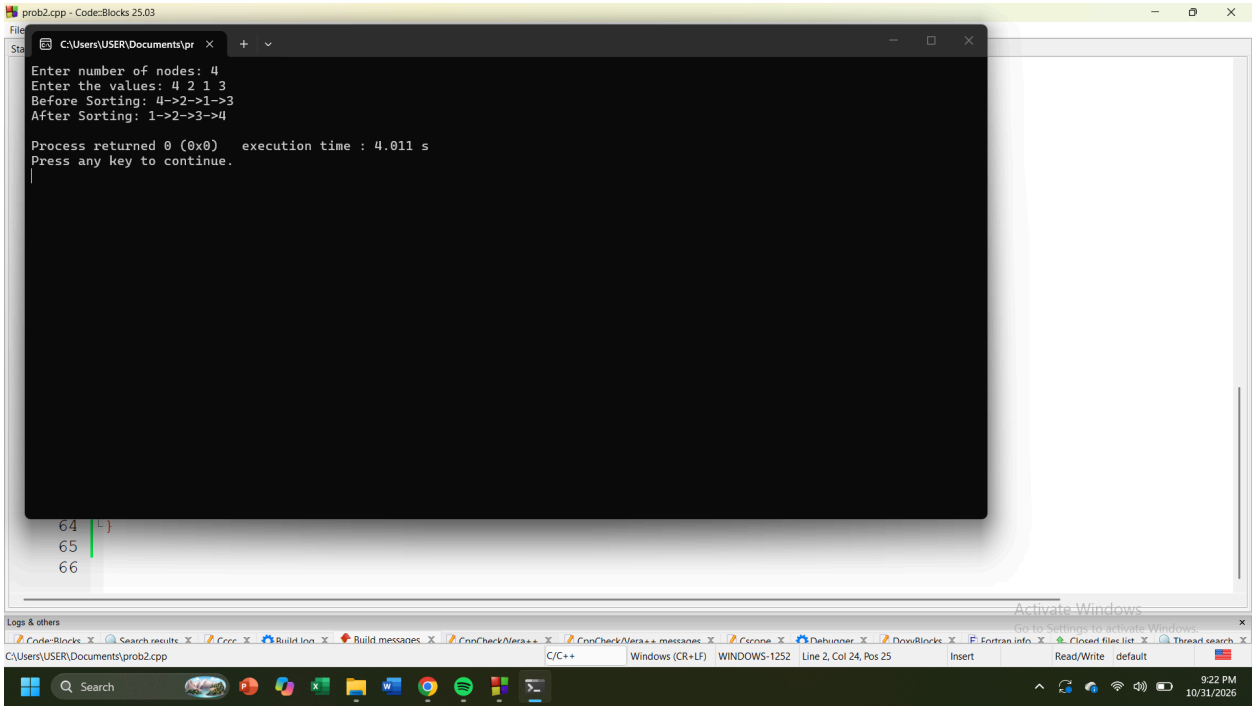

prob2.cpp - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
Start here temper.cpp prob2.cpp prob3.cpp prob4.cpp
#include <bits/stdc++.h>
using namespace std;
struct Node {
int data;
Node *next;
};
Node* createNode(int val) {
Node *newNode = new Node();
newNode->data = val;
newNode->next = NULL;
return newNode;
}
void traversal(Node *cur) {
while (cur != NULL) {
cout << cur->data;
cur = cur->next;
if (cur != NULL)
cout << "->";
}
}
int main() {
int n;
cout << "Enter number of nodes: ";
cin >> n;
Logs & others
C:\Users\USER\Documents\prob2.cpp
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 2, Col 24, Pos 25
Insert
Read/Write
default
Search
9:20 PM
10/31/2026


prob2.cpp - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
Start here temper.cpp prob2.cpp prob3.cpp prob4.cpp
}
int main() {
int n;
cout << "Enter number of nodes: ";
cin >> n;
Node *head = NULL, *cur = NULL;
cout << "Enter the values: ";
for (int i = 0; i < n; i++) {
int val;
cin >> val;
Node *temp = createNode(val);
if (head == NULL) {
head = temp;
cur = temp;
} else {
cur->next = temp;
cur = cur->next;
}
}
cout << "Before Sorting: ";
traversal(head);
cout << endl;
int arr[10000];
cur = head;
Logs & others
C:\Users\USER\Documents\prob2.cpp
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 2, Col 24, Pos 25
Insert
Read/Write
default
Search
9:21 PM
10/31/2026



```
    }
cout << "Before Sorting: ";
    traversal(head);
    cout << endl;
    int arr[10000];
    cur = head;
    int i = 0;
    while (cur != NULL) {
        arr[i] = cur->data;
        cur = cur->next;
        i++;
    }
    sort(arr, arr + n);
    cur = head;
    for (int i = 0; i < n; i++) {
        cur->data = arr[i];
        cur = cur->next;
    }
    cout << "After Sorting: ";
    traversal(head);
    cout << endl;
    return 0;
}
```

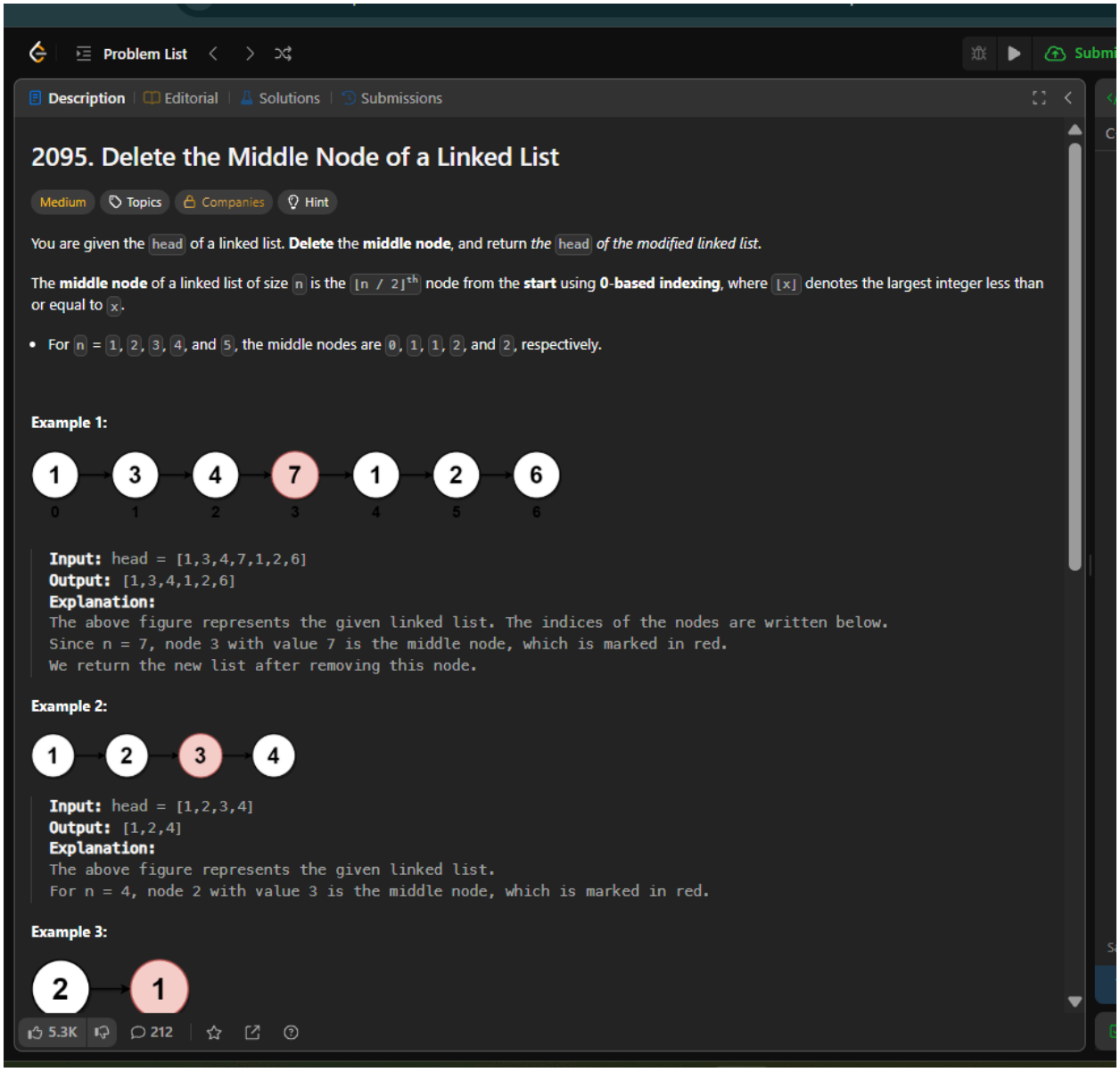
Output:



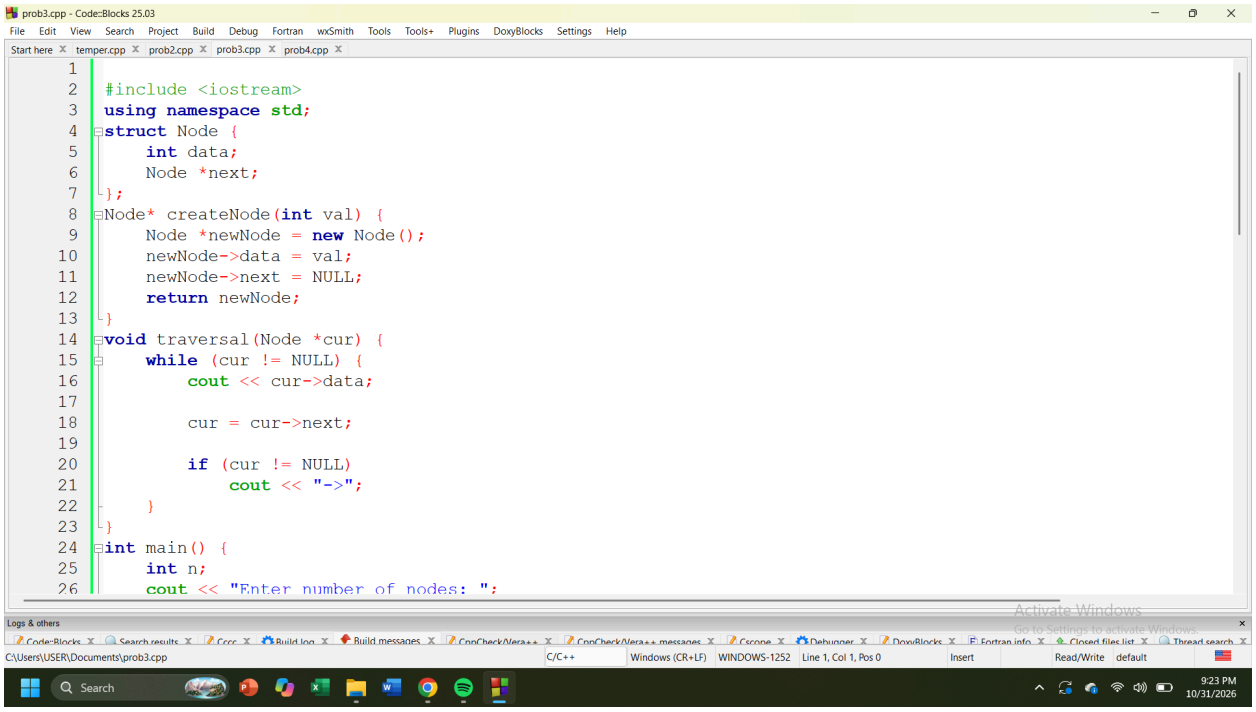
```
Enter number of nodes: 4
Enter the values: 4 2 1 3
Before Sorting: 4->2->1->3
After Sorting: 1->2->3->4

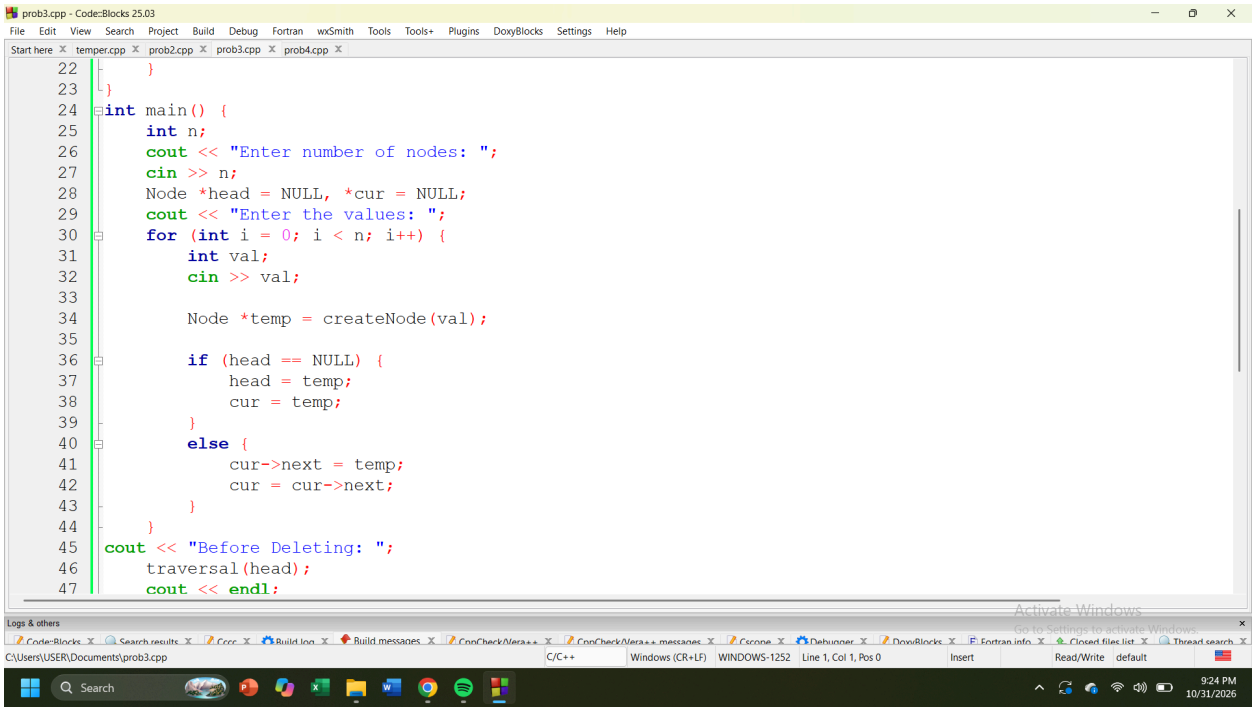
Process returned 0 (0x0)   execution time : 4.011 s
Press any key to continue.
```

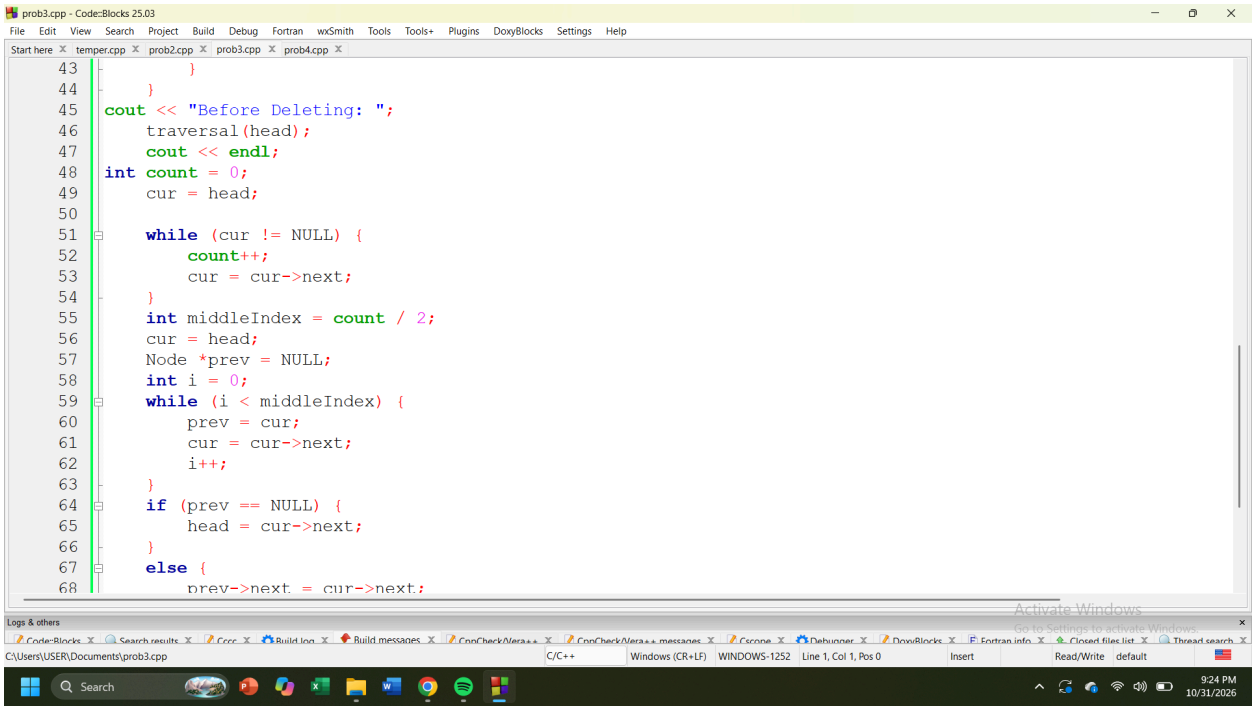
Q3:


Problem List
Description
Editorial
Solutions
Submissions
2095. Delete the Middle Node of a Linked List
Medium
Topics
Companies
Hint
You are given the head of a linked list. Delete the middle node, and return the head of the modified linked list.
The middle node of a linked list of size n is the ⌊n / 2⌋th node from the start using 0-based indexing, where ⌊x⌋ denotes the largest integer less than or equal to x.
For n = 1, 2, 3, 4, and 5, the middle nodes are 0, 1, 1, 2, and 2, respectively.
Example 1:
1 3 4 7 1 2 6
0 1 2 3 4 5 6
Input: head = [1,3,4,7,1,2,6]
Output: [1,3,4,1,2,6]
Explanation:
The above figure represents the given linked list. The indices of the nodes are written below.
Since n = 7, node 3 with value 7 is the middle node, which is marked in red.
We return the new list after removing this node.
Example 2:
1 2 3 4
Input: head = [1,2,3,4]
Output: [1,2,4]
Explanation:
The above figure represents the given linked list.
For n = 4, node 2 with value 3 is the middle node, which is marked in red.
Example 3:
2 1
5.3K
212

Codes:

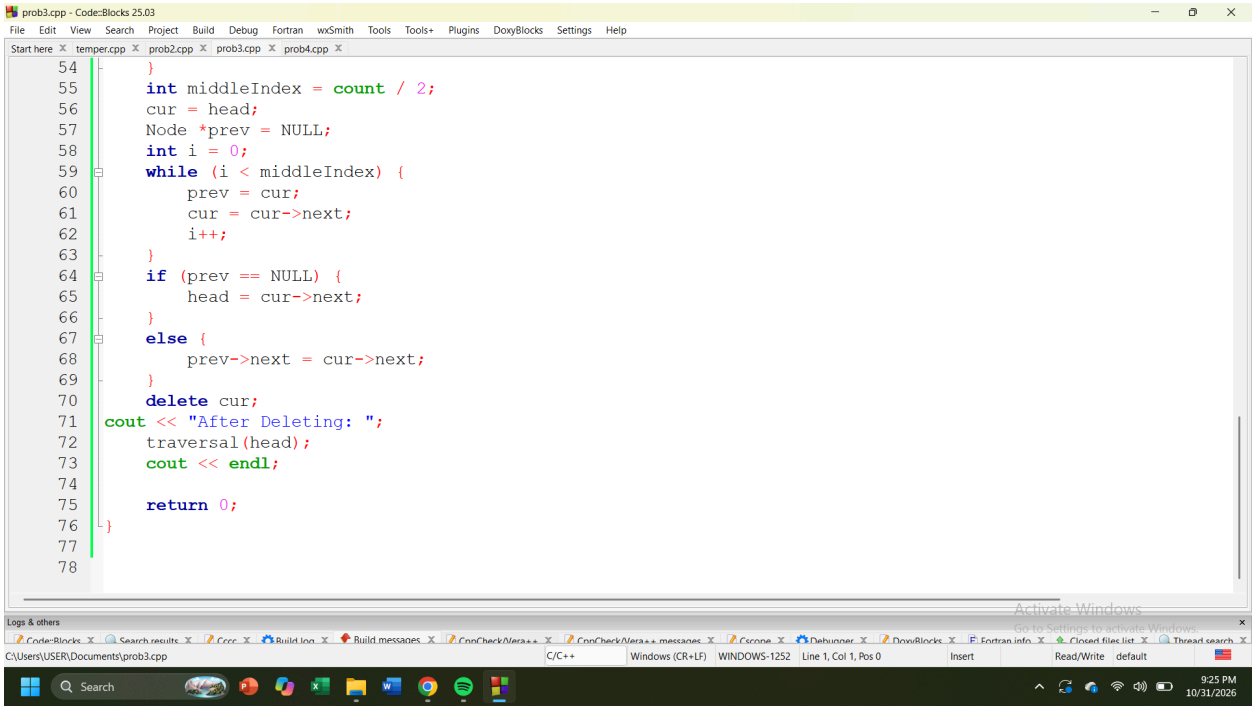

prob3.cpp - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
Start here | temper.cpp | prob2.cpp | prob3.cpp | prob4.cpp
#include <iostream>
using namespace std;
struct Node {
int data;
Node *next;
};
Node* createNode(int val) {
Node *newNode = new Node();
newNode->data = val;
newNode->next = NULL;
return newNode;
}
void traversal(Node *cur) {
while (cur != NULL) {
cout << cur->data;
cur = cur->next;
if (cur != NULL)
cout << "->";
}
}
int main() {
int n;
cout << "Enter number of nodes: ";
Logs & others
C:\Users\USER\Documents\prob3.cpp
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 1, Col 1, Pos 0
Insert
Read/Write
default
Search
9:23 PM
10/31/2026


prob3.cpp - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
Start here | temper.cpp | prob2.cpp | prob3.cpp | prob4.cpp
}
}
int main() {
int n;
cout << "Enter number of nodes: ";
cin >> n;
Node *head = NULL, *cur = NULL;
cout << "Enter the values: ";
for (int i = 0; i < n; i++) {
int val;
cin >> val;
Node *temp = createNode(val);
if (head == NULL) {
head = temp;
cur = temp;
}
else {
cur->next = temp;
cur = cur->next;
}
}
cout << "Before Deleting: ";
traversal(head);
cout << endl;
Logs & others
C:\Users\USER\Documents\prob3.cpp
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 1, Col 1, Pos 0
Insert
Read/Write
default
Search
9:24 PM
10/31/2026



```
        }
    }
cout << "Before Deleting: ";
    traversal(head);
    cout << endl;
int count = 0;
    cur = head;

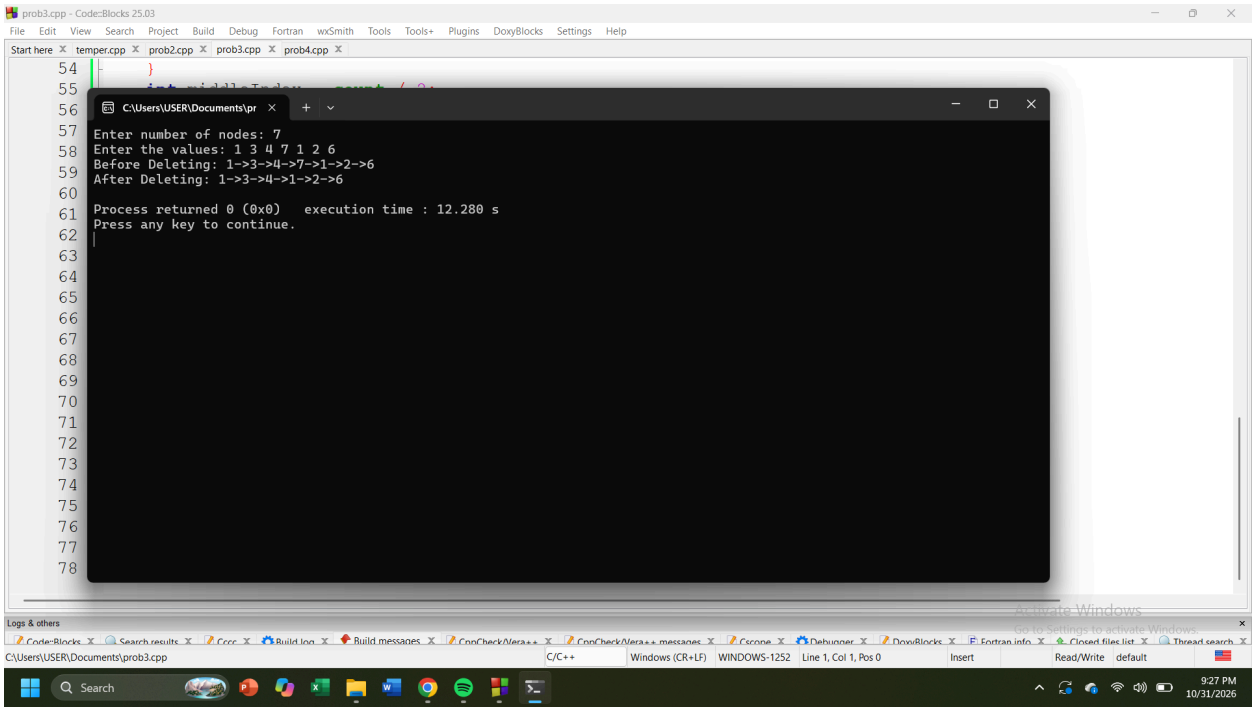
    while (cur != NULL) {
        count++;
        cur = cur->next;
    }
    int middleIndex = count / 2;
    cur = head;
    Node *prev = NULL;
    int i = 0;
    while (i < middleIndex) {
        prev = cur;
        cur = cur->next;
        i++;
    }
    if (prev == NULL) {
        head = cur->next;
    }
    else {
        prev->next = cur->next;
```

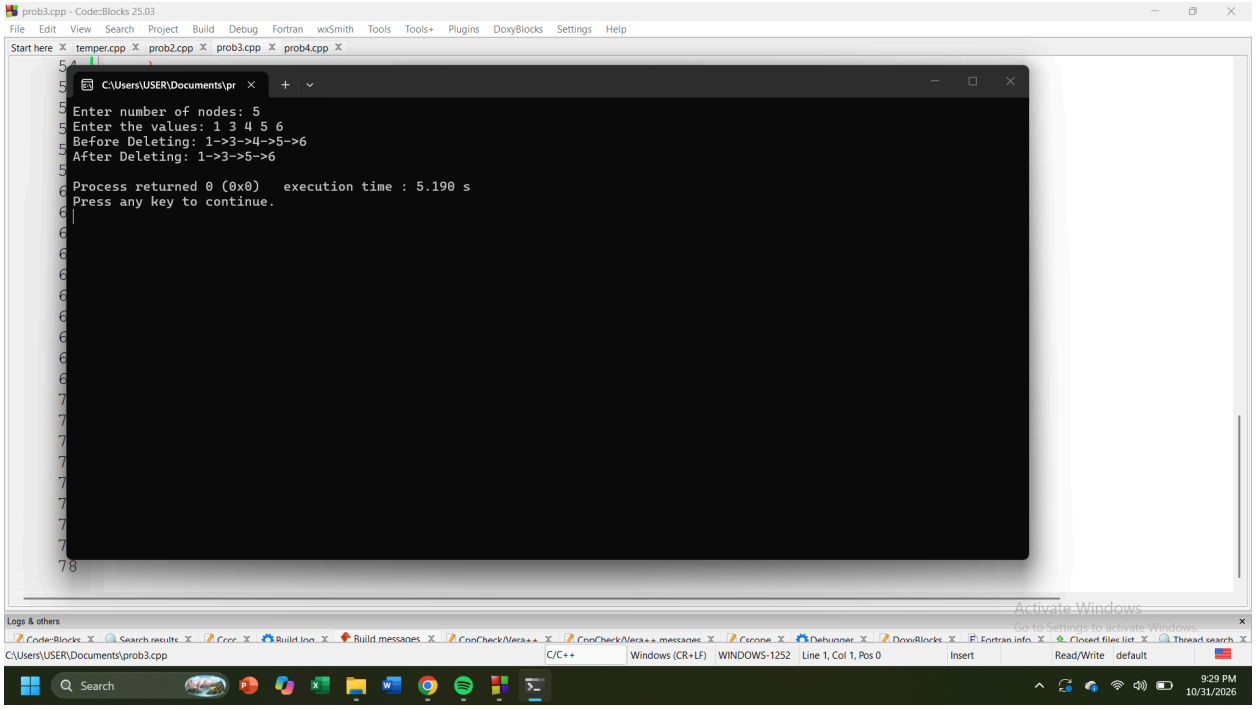


```
    }
    int middleIndex = count / 2;
    cur = head;
    Node *prev = NULL;
    int i = 0;
    while (i < middleIndex) {
        prev = cur;
        cur = cur->next;
        i++;
    }
    if (prev == NULL) {
        head = cur->next;
    }
    else {
        prev->next = cur->next;
    }
    delete cur;
cout << "After Deleting: ";
    traversal(head);
    cout << endl;

    return 0;
}
```

Output:


prob3.cpp - Code::Blocks 25.03
C:\Users\USER\Documents\pr
Enter number of nodes: 7
Enter the values: 1 3 4 7 1 2 6
Before Deleting: 1->3->4->7->1->2->6
After Deleting: 1->3->4->1->2->6
Process returned 0 (0x0) execution time : 12.280 s
Press any key to continue.


prob3.cpp - Code::Blocks 25.03
C:\Users\USER\Documents\pr
Enter number of nodes: 5
Enter the values: 1 3 4 5 6
Before Deleting: 1->3->4->5->6
After Deleting: 1->3->5->6
Process returned 0 (0x0) execution time : 5.190 s
Press any key to continue.

Q4:

1139. Largest 1-Bordered Square

Medium | Topics | Companies | Hint

Given a 2D `grid` of `0`s and `1`s, return the number of elements in the largest **square** subgrid that has all `1`s on its **border**, or `0` if such a subgrid doesn't exist in the `grid`.

Example 1:

> **Input:** grid = [[1,1,1],[1,0,1],[1,1,1]]
> **Output:** 9

Example 2:

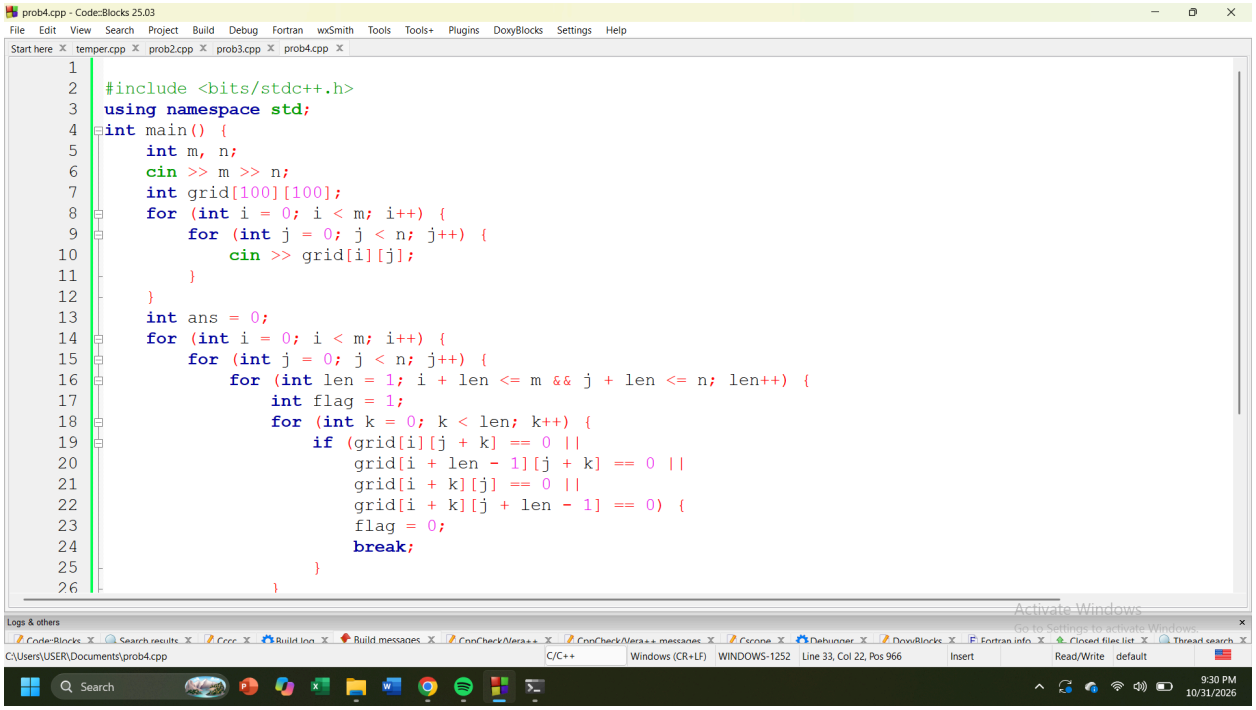
> **Input:** grid = [[1,1,0,0]]
> **Output:** 1

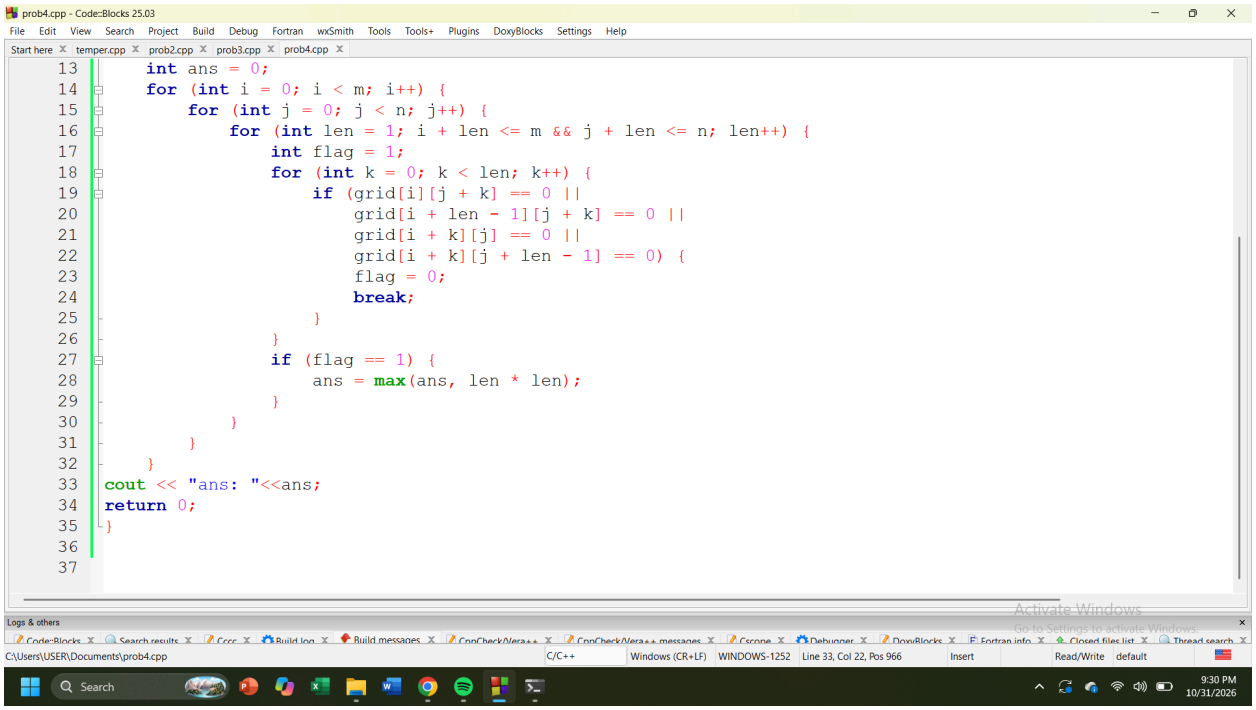
Constraints:

- `1 <= grid.length <= 100`
- `1 <= grid[0].length <= 100`
- `grid[i][j]` is `0` or `1`

Accepted 35,385 /67.1K | Acceptance Rate 52.7%

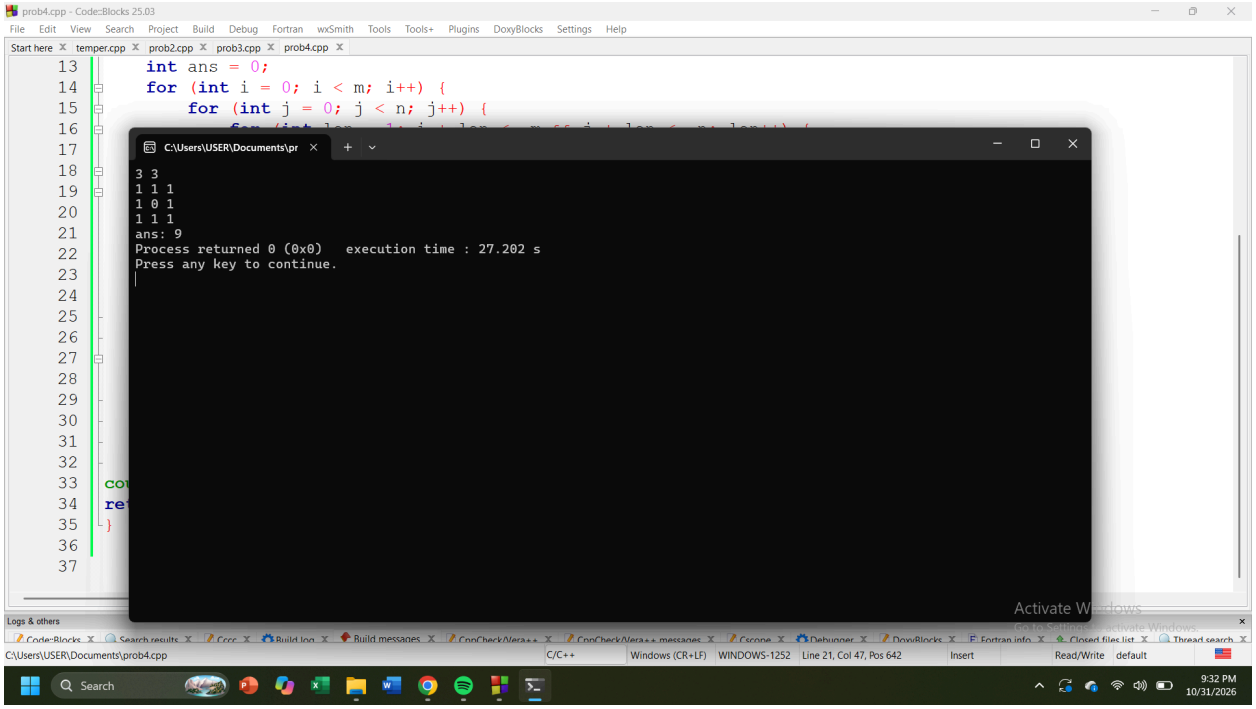
Codes:





```cpp
    int ans = 0;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            for (int len = 1; i + len <= m && j + len <= n; len++) {
                int flag = 1;
                for (int k = 0; k < len; k++) {
                    if (grid[i][j + k] == 0 ||
                        grid[i + len - 1][j + k] == 0 ||
                        grid[i + k][j] == 0 ||
                        grid[i + k][j + len - 1] == 0) {
                        flag = 0;
                        break;
                    }
                }
                if (flag == 1) {
                    ans = max(ans, len * len);
                }
            }
        }
    }
cout << "ans: "<<ans;
return 0;
}
```

Output:



```
3 3
1 1 1
1 0 1
1 1 1
ans: 9
Process returned 0 (0x0)   execution time : 27.202 s
Press any key to continue.
```

University of Information Technology & Sciences (UITS)
Department of Computer Science & Engineering
BSc. in CSE — 2nd Year, 3rd Semester | Autumn 2026
CSE0613212: Data Structure and Algorithms 1 Lab

Skill Mapping and Justification Form

CO–PO Mapping, Complex Engineering Problem (CEP) and Knowledge Profile (KP) Justification Report

Assignment/Project Title: "Problem Solving Using Arrays and Linked Lists in C++"

No.	COs	POs	CP/WP	CA/EA	KP/WK
CO1	Apply basic array operations and algorithmic techniques to solve programming problems.	1	-	-	1,2,3
CO2	Implement different linked list operations using nodes and pointers.	2	1,2	-	2,3,4
CO3	Develop and analyze solutions using arrays, linked lists and sorting techniques.	3	1,2,3	-	3,4,5

**COs- Course Outcome; POs -Program Outcome; CP/WP- Complex Problems; CA/EA- Complex Activities; KP/WK- Knowledge Profile.*

Justification of Program Outcome (PO) through Course Outcome (COs)

COs No.	POs	Justification
CO1	**PO1 (Engineering Knowledge)**	This assignment applies basic programming and data structure knowledge to solve problems using arrays. In the Daily Temperatures problem, temperature values are stored in an array and compared to find the number of days until a warmer temperature. In the Largest 1-Bordered Square problem, a two-dimensional array is used to check the boundary elements of possible squares. These tasks show the use of logical thinking, indexing and array operations.
CO2	**PO2 (Problem Analysis)**	Students analyze how linked list nodes are connected through pointers and determine the steps required for each operation. In the linked list tasks, students create and traverse nodes, update values and delete the middle node by locating its position and changing the links. The problems require identifying the input, understanding the required output and selecting the correct steps to solve the problem.
CO3	**PO3 (Design/Development of Solutions)**	Students develop C++ programs to solve the given data structure problems. They use arrays to store and process values, implement linked list operations and apply sorting to arrange data. The programs are tested with sample inputs to check whether the expected outputs are produced. The selected data structures and algorithms are used to build solutions according to the problem requirements.

Justification of Complex Engineering Problem (P) and Knowledge Profile (K)

Attributes	Justification
P1: Depth of Knowledges	Solving these problems requires knowledge of arrays, two-dimensional arrays, pointers, nodes and linked lists. Students use these concepts to store and process data, traverse nodes, update values and perform deletion. They also apply sorting and logical comparisons to obtain the required results. The assignment requires students to understand and apply data structure concepts instead of only memorizing definitions.
P2: Range of Conflicting Requirements	Students need to consider different ways of storing and processing data. Arrays allow direct access through indexes, while linked lists require traversal using pointers. The choice of a suitable method depends on the problem requirements. Students also consider correctness, time required for processing and memory usage while developing a solution.
P3: Depth of Analysis Required	The assignment requires students to analyze each problem, identify the input and expected output, and prepare a suitable algorithm. For linked list deletion, they must find the middle node and correctly update the previous node's link. For array problems, they compare elements and check square boundaries systematically. The program is then tested to verify the result.
P4: Familiarity of Issues	Not applicable. This attribute is not confirmed in the available DSA-1 Lab course outline, so it is not claimed for this assignment.
P5: Extent of applicable codes	Not applicable. No applicable codes or standards are specified in the assignment information available.
P6: Extent of stakeholder involvement and conflicting requirements	Not applicable. The selected programming problems do not specify stakeholder involvement, so this attribute is not claimed.
P7: Interdependence	Not applicable. The official course mapping for this attribute was not provided, so it is not claimed for this assignment.

Justification of Knowledge Profile (KP)

Knowledge Profile	Justification
K1 – Mathematics and Engineering Fundamentals	Students use logical reasoning, indexing, counting and comparison operations while processing array elements and linked list nodes. These fundamentals help them identify positions, compare values and calculate the required results in the given problems.
K2 – Conceptual Understanding of Engineering Sciences	The assignment requires understanding how arrays store data in indexed locations and how linked list nodes are connected through pointers. Students use the concepts of traversal, updating, deletion and sorting to understand how each program works.